

MODULE 2 ASSIGNMENT

Software Development Life Cycle (SDLC) in the Age of AI

Course Name: AI Driven SQA: Manual & Automation Testing

Name: Md. Mahfuzur Rahman

Batch No: 21

Q1. Draw the SDLC lifecycle diagram and briefly explain the workflow?

Answer: Software Development Life Cycle (SDLC) is a systematic process which provides a structured approach that helps software teams develop high quality software.

The six phases of the SDLC are:

1. Planning and Requirement Analysis
2. System Design
3. Implementation / Development
4. Testing
5. Deployment
6. Maintenance

SDLC Lifecycle Diagram:

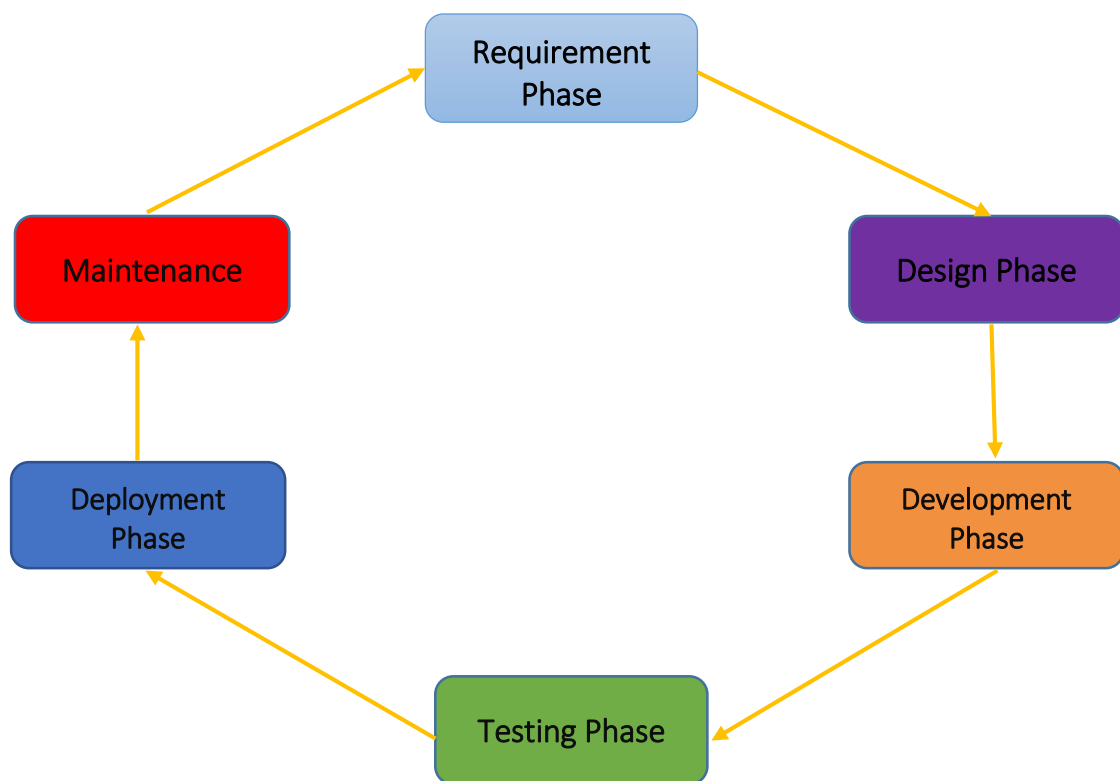


Figure 1: Software Development Life Cycle (SDLC) diagram.

Brief Explanation of the SDLC Workflow:

Requirement Analysis Phase: This is the first and one of the most important phases of SDLC. In this phase, the development team identifies the purpose of the software and collects requirements from customers, users and other stakeholders.

Activities:

- ❖ Meet with clients and stakeholders
- ❖ Gather business requirements
- ❖ Understanding user needs
- ❖ Define functional, non-functional requirements
- ❖ Estimating cost and time
- ❖ Identify system constraints
- ❖ Prepare BRD, SRS
- ❖ Review and approved requirements

Example: For an e-commerce application, requirements may include user registration, product search, shopping cart, online payment, product reviews.

Design Phase: After requirements are approved, the solution is designed. The Design Phase defines how the system will be built. Developers first create the architecture and technical blueprint.

Activities:

- ❖ High-Level Design (HLD)
- ❖ Low-Level Design (LLD)
- ❖ Database Design
- ❖ API Design
- ❖ UI/UX Design

Example: The team may design separate modules for users, products, shopping carts, payments, and orders.

Implementation (Development) Phase: This is the phase where programmers write the actual source code based on the approved design.

Activities:

- ❖ Writing source code
- ❖ Creating database tables
- ❖ Developing API's
- ❖ Code review
- ❖ Unit testing
- ❖ CI/CD Integration

Testing Phase: Testing ensure that the software works according to the requirements and is free from critical bugs or defects before release.

Activities:

- ❖ Test planning
- ❖ Test case design
- ❖ Test execution
- ❖ Defect reporting
- ❖ Regression testing
- ❖ Retesting
- ❖ Automation testing
- ❖ Performance testing
- ❖ Security testing
- ❖ User acceptance testing

Example: For an e-commerce application, testers may verify whether a customer can successfully add a product to the cart and complete payment.

Deployment Phase: After successful testing, the software is released to the production environment so that real users can use it.

Activities:

- ❖ Production release
- ❖ Database migration
- ❖ Configuration updates
- ❖ Smoke testing
- ❖ Monitoring the application
- ❖ Rollback planning

Maintenance Phase: Maintenance begins after deployment to keep the application functional, secure, and up to date.

Activities:

- ❖ Fixing bugs
- ❖ Improving performance
- ❖ Adding new features
- ❖ Updating security
- ❖ Customer support
- ❖ Monitoring

Example: An e-commerce company fix bugs, may add a new payment method based on customer feedback.

Q2. Practical Ai Exercise: For this practical exercise, I choose “Claude” as the AI tool.

Step 1: Generate five user stories for an E-commerce application

Act as an expert Senior QA Engineer / QA Automation Engineer.
 Generate five user stories for an E-commerce application.
 My mentor provide me a Practical AI Exercise task.
 - Use standard QA terminology (CSTE/ISTQB standard).
 - The output must be logically structured, high-priority, and actionable.
 Provide the response in a Markdown table with proper appropriate columns.

E-Commerce Application — User Stories (QA Perspective)

Below are five high-priority user stories written in standard Agile format, annotated with QA-relevant fields (Priority, Acceptance Criteria, and applicable ISTQB/CSTE testing techniques) to support test case design and traceability.

ID	User Story	Acceptance Criteria (Given/When/Then)	Priority (MoSCoW)	Suggested Test Techniques (ISTQB)
US-01	As a registered customer , I want to search for products by keyword , so that I can quickly find items I intend to purchase .	Given I am on the homepage or search bar, When I enter a valid keyword and press Enter, Then relevant products matching the keyword are displayed within 2 seconds, with an appropriate "No results found" message for invalid/unmatched keywords.	Must Have	Equivalence Partitioning, Boundary Value Analysis (empty/special chars), Performance Testing
US-02	As a customer , I want to add products to my shopping cart , so that I can purchase multiple items in a single checkout .	Given a product is in stock, When I click "Add to Cart," Then the item appears in the cart with correct price, quantity, and updated cart total; out-of-stock items should display a disabled/greyed "Add to Cart" button.	Must Have	Functional Testing, State Transition Testing (cart states), Boundary Value Analysis (max quantity limits)
US-03	As a customer , I want to securely complete payment during checkout , so that I can confirm my order without risk of data loss or fraud .	Given valid cart items and shipping details, When I submit valid payment information, Then the transaction is processed successfully, a confirmation number is generated, and sensitive data (card number/CVV) is never stored in plain text or displayed post-submission.	Must Have	Security Testing, Negative Testing (invalid card/expired card), Integration Testing (payment gateway)
US-04	As a registered user , I want to reset my forgotten password , so that I can regain access to my account without contacting support .	Given I click "Forgot Password" and enter a registered email, When I click the reset link sent to my inbox within a valid time window, Then I can set a new password that meets complexity rules; expired/invalid links must display an appropriate error.	Should Have	Negative Testing, Boundary Value Analysis (link expiry time), Usability Testing
US-05	As a customer , I want to view real-time order tracking status , so that I can stay informed about my delivery timeline .	Given an order has been placed, When I navigate to "My Orders," Then the current status (Processing/Shipped/Out for Delivery/Delivered) is displayed accurately and updates without requiring a page refresh (or via defined refresh interval).	Could Have	Functional Testing, Compatibility Testing (cross-browser/device), Regression Testing

Figure 1: AI-generated user stories for the e-commerce application
[Drive Screenshot link](#)

Step 2: Convert one user story into at least five acceptance criteria

Selected user story: US-03 As a customer, I want to securely complete payment during checkout, so that I can confirm my order without risk of data loss or fraud.

Now Convert user story of US-03 (As a customer, I want to securely complete payment during checkout, so that I can confirm my order without risk of data loss or fraud) into at least five acceptance criteria.

US-03: Secure Payment During Checkout — Acceptance Criteria

User Story: As a customer, I want to securely complete payment during checkout, so that I can confirm my order without risk of data loss or fraud.

Below are the detailed acceptance criteria, written in **Given-When-Then (Gherkin)** format, covering functional, security, negative, and integration scenarios as per ISTQB requirements-based test design.

AC ID	Acceptance Criteria (Given/When/Then)	Test Category
AC-01	Given I have items in my cart and valid shipping details entered, When I enter valid payment information (card number, expiry, CVV) and click "Pay Now," Then the payment is processed successfully, an order confirmation number is generated, and I am redirected to an order success page.	Functional / Positive
AC-02	Given I am on the payment page, When I enter an invalid or expired card number, Then the system displays a clear, specific error message (e.g., "Card expired" or "Invalid card number") and does not process the transaction.	Negative Testing
AC-03	Given I have submitted my payment details, When the transaction is processed, Then sensitive data (full card number, CVV) is transmitted only over an encrypted (HTTPS/TLS) connection and is never stored in plain text in application logs or the database.	Security Testing
AC-04	Given I have entered valid payment details, When I click "Pay Now" more than once (e.g., due to slow response or double-click), Then the system prevents duplicate transactions/double charges and processes only a single valid order.	Functional / Edge Case
AC-05	Given the payment gateway (third-party service) is unreachable or times out during transaction processing, When I attempt to complete payment, Then the system displays a graceful error/timeout message, the cart items remain intact, and no partial or duplicate charge occurs.	Integration / Reliability Testing
AC-06	Given I have completed a successful payment, When the transaction is confirmed, Then I receive an order confirmation via email/SMS containing order ID and masked payment details (e.g., "**** * 1234") — never the full card number.	Functional / Data Privacy
AC-07	Given I attempt checkout after my session has been idle beyond the defined timeout threshold, When I try to submit payment, Then the system requires re-authentication before processing the transaction, preventing unauthorized use of an abandoned session.	Security / Session Management

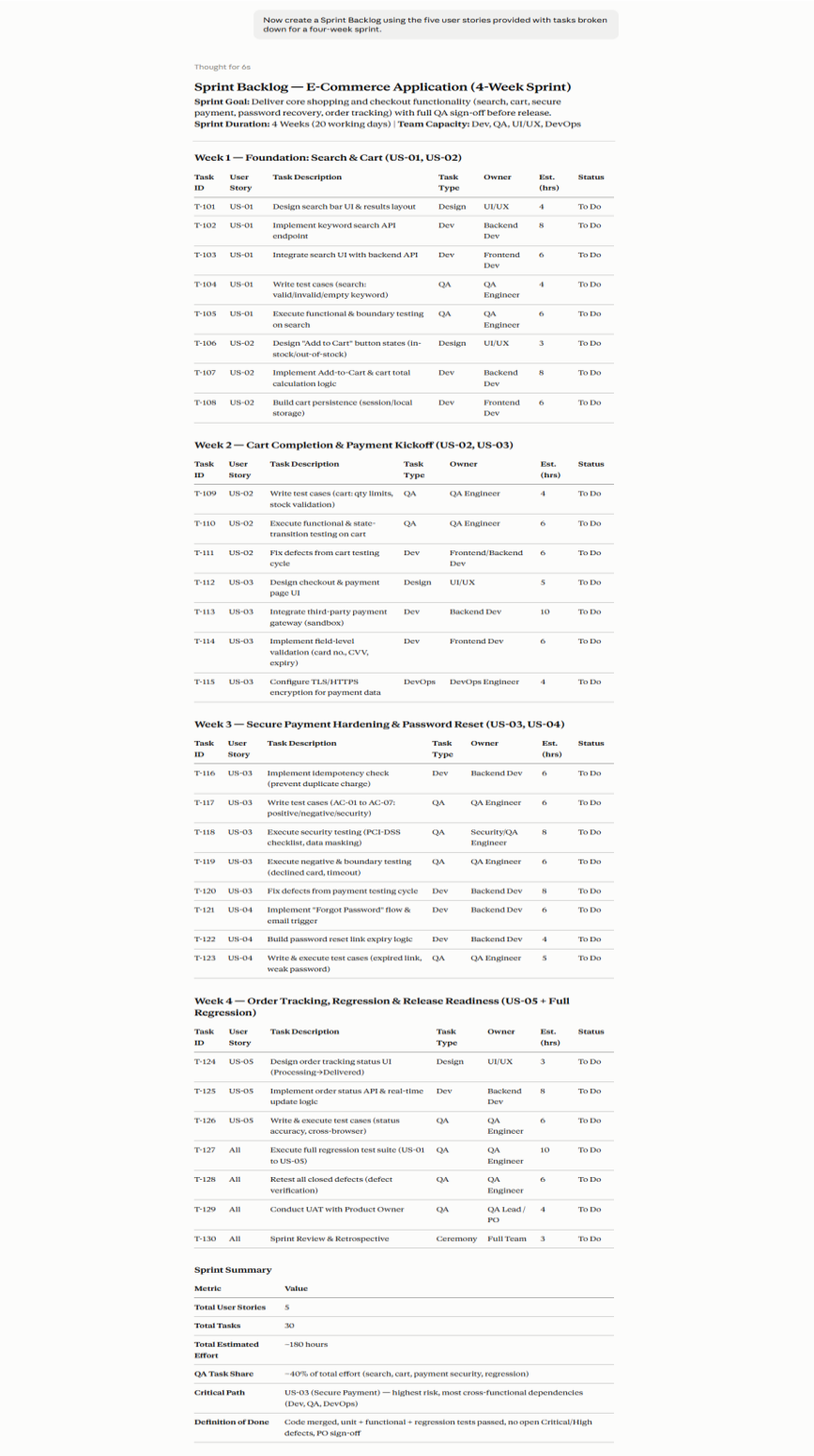
QA Notes:

- **AC-03, AC-06, AC-07** are critical for **PCI-DSS compliance** validation — these should be flagged as **high-risk security test cases** in your test plan.
- **AC-04 and AC-05** address **non-functional reliability concerns** (idempotency and fault tolerance), which are often overlooked but essential for payment systems.
- Each AC above can be further decomposed into **multiple test cases** (e.g., AC-02 alone could generate test cases for expired card, invalid CVV format, incorrect card number length, unsupported card type, etc.) using **Equivalence Partitioning** and **Boundary Value Analysis**.

Figure 2: AI-generated acceptance criteria

[Drive Screenshot link](#)

Step 3: Ask the AI to create a Sprint Backlog



Step 4: Review and Improve the AI-generated output

What the AI did well:

- Generate structured output following standard Agile formats
- Improved productivity by reduced the time required to prepare initial requirements and planning requirements
- Understand basic e-commerce requirements and created relevant user stories
- Produced acceptance criteria that were mostly testable
- Provided a reasonable starting point for sprint planning with tasks
- AI provided output can be used as a draft that a software development team can review, modify, and improve

What corrections were made:

- Made the user stories more specific
- Added missing edge cases in acceptance criteria
- Recognized task priorities
- Clarified ambiguous user roles
- Added missing cart functions such as removing products
- Added priorities to Sprint Backlog items
- Removed unnecessary tasks
- Added business requirements like: security standards, that AI overlooked

Why human review is still necessary:

- AI-generated information may be incomplete, inaccurate, or unsuitable for a particular project.
- A real software project may have some business rules, customer requirements that AI does not know so AI may not understand business context.
- Security requires human judgment to handle security tasks carefully.
- AI-generated user story sometimes not fulfill the actual customer wants so human review is must require.
- Some AI suggestion may be technically difficult or expensive with the existing system.
- The main thing is AI is an assistant not the final decision maker. So final responsibility belongs to human.

THE END